SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5802	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3
Weightage:	40% of CIA	**Total Marks:**	100
CO 3 Statement:	Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation		
Student Name:	K.Yalla Reddy	**Reg. No.:**	192465047
Section / Batch:	2024	**Date:**	13-08-2026

Instructions to Students

- Perform vulnerability detection and classify the identified vulnerabilities based on their severity levels.
- Conduct severity analysis using appropriate metrics such as CVSS and document the impact of each vulnerability.
- Design and simulate a Security Verification Workflow covering detection, remediation, re-testing, and verification.
- Evaluate security and system performance before and after mitigation, and include relevant results, screenshots, and observations.

Vulnerability Detection Conceptual Understanding Q1 Q1

–100

Code of Conduct

I

K.Yalla Reddy(192465047) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Proficient	Developing	Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or

misinterpretation of the problem

Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

CSA5802 - DEVSECOPS ENGINEERING AND APPLICATION SECURITY

CO3- ASSESSMENT TASK 3:

VULNERABILITY DETECTION, SEVERITY ANALYSIS, SECURITY VERIFICATION WORKFLOW SIMULATION AND PERFORMANCE EVALUATION

NAME: YALLA REDDY.K

REG.NO:192465047

SHIFT-LEFT SECURE SDLC PLATFORM WITH POLICY-AS-CODE GATEKEEPING

CSA5802 – DEVSECOPS ENGINEERING AND APPLICATION SECURITY

Assessment Tool 3

CO Assessed: CO3 – Precision (BL1, BL2, BL3)
CO Statement: Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation
Weightage: 40% of CIA
Total Marks: 100

Student Name: K. Yalla Reddy
Register Number: 192465047
Section / Batch: 2024
Date: 13-08-2026

ABSTRACT

Modern software development requires security to be integrated into every stage of the Software Development Life Cycle (SDLC). Traditional security practices often perform security testing near the end of development, which can result in increased remediation cost and delayed deployment. The Shift-Left Security approach addresses this problem by introducing security controls during the early stages of software development.

This project proposes a **Shift-Left Secure SDLC Platform with Policy-as-Code Gatekeeping**. The platform integrates automated security checks into the software development and CI/CD workflow. It performs source-code vulnerability detection, dependency analysis, secret detection, severity classification, policy evaluation, security verification, and deployment gatekeeping.

A simulated platform is developed using Python to demonstrate how security policies can automatically determine whether a software build should be allowed or blocked. Policies are defined as code and evaluated against detected vulnerabilities. For example, a build containing a Critical vulnerability or an exposed secret is automatically blocked, while a build satisfying the defined security requirements is allowed to proceed.

The project also implements a security verification workflow consisting of detection, severity analysis, remediation, re-testing, policy evaluation, and final verification. Performance metrics are compared before and after mitigation to evaluate the impact of security controls.
The proposed platform demonstrates how automated security policies can provide consistent and repeatable security enforcement while supporting rapid software delivery.

Keywords: DevSecOps, Shift-Left Security, Secure SDLC, Policy-as-Code, SAST, Vulnerability Detection, CI/CD, Security Gate, CVSS, Security Verification.

CHAPTER 1 – INTRODUCTION

1.1 Background

Software applications are increasingly developed using agile methodologies, continuous integration, and continuous delivery. These approaches enable organizations to release software rapidly. However, rapid development can also introduce security vulnerabilities when security is treated as a separate activity.

Traditional Software Development Life Cycle models often perform security testing after most development activities have been completed. Discovering a vulnerability at this stage can require significant changes to the application and can delay deployment.

DevSecOps addresses this problem by integrating security into DevOps processes.

The **Shift-Left Security** approach moves security activities toward the early stages of the SDLC. Developers can identify vulnerabilities while writing and committing code instead of waiting until the application reaches production.

Policy-as-Code further improves this process by representing security requirements as machine-readable policies. These policies can automatically determine whether a software build satisfies security requirements.

1.2 Problem Statement

Software projects may contain vulnerabilities such as insecure coding practices, vulnerable dependencies,

hardcoded secrets, and inadequate security configurations.

Manual security reviews may be inconsistent and time-consuming.

The problem addressed by this project is:

How can security vulnerabilities be detected early and automatically enforced through Policy-as-Code rules so that insecure software builds are prevented from progressing through the SDLC?

1.3 Objectives

The main objectives are:

1. To design a Shift-Left Secure SDLC Platform.
2. To integrate security checks into early development stages.
3. To simulate vulnerability detection.
4. To classify vulnerabilities based on severity.
5. To apply CVSS-based severity analysis.
6. To detect vulnerable dependencies.
7. To identify exposed secrets.
8. To implement Policy-as-Code security rules.
9. To automatically allow or block software builds.
10. To simulate remediation and re-testing.
11. To evaluate security before and after mitigation.
12. To evaluate the performance of the proposed platform.
13. To demonstrate automated security gatekeeping in DevSecOps.

1.4 Scope

The project focuses on a controlled and simulated Secure SDLC

environment. The scope includes:

- Source-code security analysis
- Dependency vulnerability detection
- Secret detection
- Vulnerability classification
- CVSS severity analysis
- Policy-as-Code rules
- Build gatekeeping
- Remediation
- Re-testing
- Security verification
- Performance evaluation
- Security reporting

The project is intended for educational and controlled testing purposes.

1.5 Significance of the Project

The proposed system is significant because it:

- Detects security issues early.
- Reduces remediation cost.
- Prevents insecure builds from progressing.
- Automates security decision-making.
- Provides consistent security enforcement.
- Supports DevSecOps principles.
- Improves software security visibility.
- Reduces dependence on manual security reviews.

CHAPTER 2 – LITERATURE AND CONCEPTUAL FOUNDATION

2.1 Secure SDLC

A Secure SDLC incorporates security activities into every phase of software

development. Typical stages include:

Planning → Design → Development → Testing → Deployment → Monitoring

Security activities are performed throughout these stages.

2.2 Shift-Left Security

Shift-Left Security means moving security activities earlier in the

SDLC. Traditional approach:

Development → Testing → Deployment → Security Testing

Shift-Left approach:

Development → Security Testing → Remediation → Testing → Deployment

The main objective is to detect vulnerabilities before they reach later stages.

2.3 DevSecOps

DevSecOps integrates development, operations, and security.

The traditional DevOps cycle can be represented as:

Plan → Code → Build → Test → Release → Deploy → Operate

DevSecOps adds automated security controls:

Plan → Secure Code → Security Scan → Secure Build → Security Gate → Deploy

2.4 Policy-as-Code

Policy-as-Code represents security and compliance requirements as machine-readable

rules. Instead of manually checking:

"No Critical vulnerabilities should exist before deployment."

The requirement can be represented programmatically:

```
IF critical_vulnerabilities > 0
THEN build = BLOCK
```

This allows security policies to be automatically evaluated.

2.5 Security Gate

A security gate is a control point in a CI/CD pipeline.

The gate evaluates security results and decides whether the build should

continue. Example:

```
Security Scan
 |
 v
Policy Evaluation
 |
 +--+--+
 | |
 PASS FAIL
 | |
Deploy Block
```

2.6 Vulnerability Severity

Vulnerabilities can be categorized using severity levels:

Severity	CVSS Range
Critical	9.0–10.0
High	7.0–8.9
Medium	4.0–6.9
Low	0.1–3.9
None	0.0

Severity classification allows organizations to prioritize remediation.

CHAPTER 3 – PROPOSED SYSTEM

3.1 System Overview

The proposed platform integrates multiple security checks into the

SDLC. Main Workflow

Developer

↓

Source Code Repository

↓

SAST
↓

Dependency Scan

↓

Secret Scan

↓

Severity Analysis

↓

Policy-as-Code Engine

↓

Security Gate

↓

Allow / Block

↓

Remediation and Re-test

↓

Deployment

3.2 System Architecture

The proposed architecture contains the following components:

1. Developer Interface
2. Source Code Repository
3. SAST Engine
4. Dependency Scanner
5. Secret Scanner
6. Vulnerability Aggregator
7. CVSS Severity Analyzer
8. Policy-as-Code Engine
9. Security Gate
10. Verification Engine
11. Reporting Module

3.3 Module 1 – Secure Code Analysis

This module analyzes source code for common security

problems. Examples include:

- SQL Injection
- Cross-Site Scripting
- Command Injection
- Insecure functions
- Hardcoded credentials
- Weak cryptographic functions

The output contains:

- Vulnerability ID
- File name
- Line number
- Vulnerability type
- Severity
- Recommendation

3.4 Module 2 – Dependency and Secret Scanning

This module checks third-party dependencies and source code for security

problems. Dependency Scanning

It identifies:

- Outdated packages
- Vulnerable libraries
- Known security issues
- Unsafe dependency versions

Secret Scanning

It detects potentially exposed:

- API keys
- Passwords
- Access tokens
- Private keys
- Database credentials

A detected secret can automatically trigger a security gate failure.

3.5 Module 3 – Policy-as-Code Gatekeeping

This is the core module of the project.
Security policies are represented as executable rules.

Example policies:

POLICY 1:
IF Critical vulnerabilities > 0
THEN BLOCK BUILD

POLICY 2:
IF High vulnerabilities > 2
THEN BLOCK BUILD

POLICY 3:
IF exposed secrets > 0
THEN BLOCK BUILD

POLICY 4:
IF security score < 80
THEN BLOCK BUILD

POLICY 5:
IF all security requirements are satisfied
THEN ALLOW BUILD

3.6 Module 4 – Security Verification and Reporting

This module performs:

- Re-testing
- Policy re-evaluation
- Verification
- Security score calculation
- Performance measurement
- Report generation

The module ensures that remediation actually resolves the identified security issue.

CHAPTER 4 – SYSTEM IMPLEMENTATION

4.1 Python Environment

The simulation can be implemented using Python.

Required libraries:

```
import time
import re
```

Optional performance monitoring can use:

```
import psutil
```

4.2 Simulated Vulnerability Dataset

The following dataset represents the output of security scanning:

```
vulnerabilities = [
 {
 "id": "V001",
 "type": "SQL Injection",
 "severity": "Critical",
 "cvss": 9.8
 },
 {
 "id": "V002",
 "type": "Cross-Site Scripting",
 "severity": "High",
 "cvss": 8.2
 },
 {
 "id": "V003",
 "type": "Weak Password Policy",
 "severity": "Medium",
 "cvss": 5.3
 },
 {
 "id": "V004",
 "type": "Hardcoded Secret",
```

```
 "severity": "Critical",
 "cvss": 9.1
 }
]
```

4.3 Vulnerability Counting

```
def count_severity(vulnerabilities):

 counts = {
 "Critical": 0,
 "High": 0,
 "Medium": 0,
 "Low": 0
 }

 for vulnerability in vulnerabilities:
 severity = vulnerability["severity"]

 if severity in counts:
 counts[severity] += 1

 return counts


results = count_severity(vulnerabilities)
print("Severity Summary")
for severity, count in results.items():
 print(severity, ":", count)
```

Expected Output

```
Severity Summary
Critical : 2
High : 1
Medium : 1
Low : 0
```

4.4 Policy-as-Code Engine

The policy engine evaluates the scan results.

```
def evaluate_policy(vulnerabilities, secrets_detected):

 critical = sum(
 1 for v in vulnerabilities
 if v["severity"] == "Critical"
 )

 high = sum(
 1 for v in vulnerabilities
 if v["severity"] == "High"
 )
```

```
 if critical > 0:
 return "BLOCK: Critical vulnerability detected"

 if high > 2:
 return "BLOCK: Too many High vulnerabilities"

 if secrets_detected:
 return "BLOCK: Exposed secret detected"

return "ALLOW: Security policy satisfied"


decision = evaluate_policy(
 vulnerabilities,
 secrets_detected=True
)

print(decision)
```

Expected Output

```
BLOCK: Critical vulnerability detected
```

4.5 Security Score

A simple security score can be calculated for the simulation.

```
def security_score(vulnerabilities):

 score = 100

 for vulnerability in vulnerabilities:

 if vulnerability["severity"] == "Critical":
 score -= 30

 elif vulnerability["severity"] == "High":
 score -= 20

 elif vulnerability["severity"] == "Medium":
 score -= 10

 elif vulnerability["severity"] == "Low":
 score -= 5

 return max(score, 0)


score = security_score(vulnerabilities)

print("Security Score:", score)
```

Expected Output

```
Security Score: 0
```

The score is illustrative and is used only to demonstrate policy evaluation.

4.6 Secure Build Gate

```
def security_gate(vulnerabilities, secrets_detected):

 decision = evaluate_policy(
 vulnerabilities,
 secrets_detected
 )

 print("Security Gate Result:", decision)

 if decision.startswith("ALLOW"):
 print("BUILD STATUS: PASSED")
 return True

 print("BUILD STATUS: BLOCKED")
 return False


security_gate(vulnerabilities, True)
```

Expected Output

```
Security Gate Result: BLOCK: Critical vulnerability
detected BUILD STATUS: BLOCKED
```

CHAPTER 5 – VULNERABILITY DETECTION AND SEVERITY ANALYSIS

5.1 Initial Security Scan

The initial scan identifies the following vulnerabilities:

ID	Vulnerability Type	CVSS	Severity
V001	SQL Injection	SAST	9.8 Critical
V002	XSS	SAST	8.2 High
V003	Weak Password Policy	SAST	5.3 Medium
V004	Hardcoded Secret	Secret Scan	9.1 Critical

5.2 Initial Security Status

Category	Count
Critical	2
High	1
Medium	1
Low	0
Total	4

The initial build should therefore be **BLOCKED**.

5.3 Impact Analysis

SQL Injection

Potential impact:

- Unauthorized database access
- Data manipulation
- Information disclosure
- Authentication bypass

XSS

Potential impact:

- Malicious script execution
- User session compromise
- Phishing
- Unauthorized browser actions

Weak Password Policy

Potential impact:

- Account compromise
- Credential guessing
- Unauthorized access

Hardcoded Secret

Potential impact:

- Unauthorized access to services
- Credential exposure
- Data leakage
- Cloud or API compromise

CHAPTER 6 – POLICY-AS-CODE GATEKEEPING

6.1 Policy Requirements

The platform defines the following security policies:

Policy ID	Rule	Action
P001	Critical vulnerabilities > 0	BLOCK
P002	High vulnerabilities > 2	BLOCK
P003	Secrets detected > 0	BLOCK
P004	Security score < 80	BLOCK

P005 All policies satisfied ALLOW

6.2 Initial Policy Evaluation

The application has:

- Critical vulnerabilities = 2
- High vulnerabilities = 1
- Exposed secrets = 1
- Security score below threshold

Therefore:

Policy Result = BLOCK

Build Result = BLOCKED

6.3 Why Gatekeeping Is Important

Without a security gate, vulnerable code could continue through the pipeline.

With Policy-as-Code:

Vulnerability Detected
↓
Policy Evaluated
↓
Security Requirement Failed
↓
BUILD BLOCKED
↓
Developer Remediation
↓
Re-test

This prevents known security issues from progressing to later environments.

CHAPTE

7.1 SQL Injection Remediation

Replace dynamically constructed SQL statements with parameterized queries.

```
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))
```

7.2 XSS Remediation

Use input validation and output encoding.

```
import html

safe_output = html.escape(user_input)
```

7.3 Weak Password Remediation

Implement stronger password requirements.

```
def strong_password(password):

 return (
 len(password) >= 12 and
 re.search(r"[A-Z]", password) and
 re.search(r"[a-z]", password) and
 re.search(r"[0-9]", password) and
 re.search(r"[^A-Za-z0-9]", password)
 )
```

7.4 Hardcoded Secret Remediation

Remove credentials from source code.

Instead of:

```
API_KEY = "my-secret-key"
```

use an environment variable:

```
import os

API_KEY = os.environ.get("API_KEY")
```

Secrets should not be committed to source-control repositories.

CHAPTER 8 – RE-TESTING AND SECURITY VERIFICATION

8.1 Re-scan

After remediation, the security scan is performed again.

Expected results:

Vulnerability	Before	After
SQL Injection	Detected	Not Detected
XSS	Detected	Not Detected
Weak Password	Detected	Not Detected
Hardcoded Secret	Detected	Not Detected

8.2 Updated Dataset

```
fixed_vulnerabilities = []
secrets_detected = False

print("Re-scanning application...")

if len(fixed_vulnerabilities) == 0:
 print("No Critical or High vulnerabilities detected.")
```

Expected Output

```
Re-scanning application...
No Critical or High vulnerabilities detected.
```

8.3 Policy Re-evaluation

```
result = evaluate_policy(
 fixed_vulnerabilities,
 secrets_detected
)

print(result)
```

Expected Output

```
ALLOW: Security policy satisfied
```

8.4 Final Security Gat

```
Security Scan
↓
No Critical Vulnerabilities
↓
No Excessive High Vulnerabilities
↓
No Exposed Secrets
↓
Security Score ≥ 80
↓
POLICY PASSED
↓
BUILD ALLOWED
```

CHAPTER 9 – PERFORMANCE EVALUATION

9.1 Purpose

Security controls should improve application security without creating unacceptable performance

overhead. The following metrics are evaluated:

- Scan execution time
- Security verification time
- CPU utilization
- Memory utilization

- Number of vulnerabilities
- Security score
- Build status

9.2 Performance Comparison

The following values demonstrate the format for recording actual experimental measurements.

Metric Before Mitigation After Mitigation
Vulnerabilities 4 0
Critical Vulnerabilities 2 0
High Vulnerabilities 1 0
Medium Vulnerabilities 1 0
Security Score 0 100
Build Status BLOCKED ALLOWED
Security Tests Passed 0% 100%
Scan Time 0.15 sec 0.17 sec

The timing values above are illustrative and should be replaced with measurements from the actual execution environment.

9.3 Performance Measurement Code

```
import time

start_time = time.perf_counter()

# Simulated security verification
for _ in range(100000):
 pass
end_time = time.perf_counter()

execution_time = end_time - start_time

print("Security verification time:",
 round(execution_time, 6), "seconds")
```

9.4 Performance Observation

The addition of security controls may slightly increase build or verification time.

However, the security benefits are significantly more important than a small amount of processing overhead.

The platform aims to maintain:

- Fast automated scans
- Early vulnerability detection

- Minimal pipeline delay
- Reliable security enforcement

CHAPTER 10 – RESULTS AND DISCUSSION

10.1 Initial Results

The initial scan identified four vulnerabilities.

Severity	Number
Critical	2
High	1
Medium	1
Low	0

Because Critical vulnerabilities and an exposed secret were detected, the Policy-as-Code engine blocked the build.

10.2 Final Results

After remediation:

Severity	Before	After
Critical	2	0
High	1	0
Medium	1	0
Low	0	0
Total	4	0

The simulated vulnerability count was reduced from **4 to 0**.

10.3 Security Improvement

The percentage reduction can be calculated as:

$$\text{Reduction} = \frac{\text{Initial Vulnerabilities - Final Vulnerabilities}}{\text{Initial Vulnerabilities}} \times 100$$

$$\frac{4-0}{4}\times100$$

$$=100\%$$

Therefore, the simulation achieved a **100% reduction in detected vulnerabilities** after remediation.

10.4 Policy Gate Results

Stage	Result
Initial Scan	Vulnerabilities detected
Severity Analysis	Critical/High issues identified
Policy Evaluation	FAILED
Build Gate	BLOCKED
Remediation	Completed
Re-test	Passed
Policy Re-evaluation	PASSED
Final Build Gate	ALLOWED

CHAPTER 11 – DEVSECOPS CI/CD INTEGRATION

The proposed platform can be integrated into a CI/CD pipeline.

Pipeline

Developer Commit
↓
Source Code Repository
↓
Build
↓
SAST Scan
↓
Dependency Scan
↓
Secret Scan
↓
Vulnerability Aggregation
↓
CVSS Analysis
↓
Policy-as-Code
↓
Security Gate
↙ ↘
BLOCK ALLOW
↓ ↓
Remediation Deploy
↓
Re-test
↓
Policy Re-check
↓
Deploy

12.1 Shift-Left Benefit

Security testing occurs immediately after code is

committed. This provides:

- Earlier vulnerability detection
- Faster remediation
- Lower remediation cost
- Better developer awareness
- Reduced production risk

12.2 Policy Enforcement Benefit

Policy-as-Code provides centralized and repeatable security

rules. For example:

No Critical Vulnerabilities
No Exposed Secrets
Maximum 2 High Vulnerabilities
Minimum Security Score = 80

These rules are automatically evaluated for every build.

CHAPTER 13 – ADVANTAGES

The proposed platform provides the following advantages:

1. Early security testing.
2. Automated vulnerability detection.
3. Automated policy enforcement.
4. Consistent security decisions.
5. Reduced human error.
6. Faster vulnerability remediation.
7. Prevention of insecure deployments.
8. Integration with CI/CD pipelines.
9. Improved security visibility.
10. Repeatable security verification.
11. Centralized security policies.
12. Better alignment with DevSecOps principles.

CHAPTER 14 – LIMITATIONS

The current implementation has the following limitations:

1. The platform is a simulated educational implementation.

2. Vulnerability data is represented using synthetic datasets.
3. CVSS scores are representative.
4. Real-world SAST and DAST tools are not fully integrated.
5. Performance results depend on the execution environment.
6. The project does not cover every application vulnerability.
7. Production deployment would require additional security controls.

CHAPTER 15 – FUTURE ENHANCEMENTS

The platform can be extended with real security tools.

Possible integrations include:

- Semgrep for SAST
- Bandit for Python security analysis
- OWASP ZAP for DAST
- Trivy for container and dependency scanning
- Gitleaks for secret detection
- SonarQube for code quality and security analysis
- GitHub Actions/GitLab CI/Jenkins for CI/CD
- OPA/Rego for advanced Policy-as-Code
- SIEM integration for security monitoring

A future implementation could use a centralized policy engine where organizational security rules are maintained independently from application code.

CHAPTER 16 – CONCLUSION

The project successfully demonstrates a **Shift-Left Secure SDLC Platform with Policy-as-Code Gatekeeping** for integrating security into the early stages of software development.
The proposed platform detects vulnerabilities through simulated SAST, dependency, and secret scanning processes. Identified vulnerabilities are classified according to severity, and CVSS-based analysis is used to prioritize security risks.

The Policy-as-Code engine acts as an automated security gate. Critical vulnerabilities, excessive high-severity vulnerabilities, and exposed secrets automatically cause the build to be blocked. This prevents insecure software from progressing toward deployment.

After remediation, the application is re-tested and the security policies are evaluated again. Once all security requirements are satisfied, the build is allowed to proceed.

The experiment demonstrates a reduction from four simulated vulnerabilities to zero after remediation. The security verification process therefore demonstrates how automated security controls can improve software security while maintaining a structured development workflow.

The project supports the core principles of DevSecOps by combining **early vulnerability detection, automated security testing, policy enforcement, remediation, verification, and continuous security improvement**.

Overall, the proposed platform satisfies the requirements of CO3 by demonstrating **vulnerability detection, severity analysis, security verification workflow simulation, policy-based gatekeeping, and performance evaluation**.

REFERENCES

1. OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks.*
2. OWASP Foundation. *OWASP Web Security Testing Guide.*
3. FIRST. *Common Vulnerability Scoring System (CVSS).*
4. NIST. *Secure Software Development Framework (SSDF).*
5. NIST. *National Vulnerability Database (NVD).*
6. NIST. *Security and Privacy Controls for Information Systems and Organizations.*
7. OWASP Foundation. *SQL Injection Prevention Cheat Sheet.*
8. OWASP Foundation. *Cross Site Scripting Prevention Cheat Sheet.*
9. OWASP Foundation. *Secrets Management Cheat Sheet.*
10. OWASP Foundation. *CI/CD Security Cheat Sheet.*
11. Kim, G., Humble, J., Debois, P., & Willis, J. *The DevOps Handbook*. IT Revolution.
12. Humble, J., & Farley, D. *Continuous Delivery*. Addison-Wesley.
13. Python Software Foundation. *Python Documentation.*
14. SANS Institute. *Secure Coding and Application Security Practices.*
15. ISO/IEC 27001. *Information Security Management Systems.*

APPENDIX A – VULNERABILITY SUMMARY

ID Vulnerability Detection Method CVSS Severity Remediation Final Status V001 SQL Injection SAST 9.8 Critical Parameterized queries Fixed V002 XSS SAST 8.2 High Output encoding Fixed V003 Weak Password Policy SAST 5.3 Medium Strong password rules Fixed V004 Hardcoded Secret Secret Scan 9.1 Critical Environment variables Fixed

APPENDIX B – POLICY RULES

```
POLICY P001
IF Critical Vulnerabilities > 0
THEN BLOCK BUILD

POLICY P002
IF High Vulnerabilities > 2
THEN BLOCK BUILD

POLICY P003
IF Exposed Secrets > 0
THEN BLOCK BUILD

POLICY P004
IF Security Score < 80
THEN BLOCK BUILD

POLICY P005
IF All Security Requirements Are Satisfied
THEN ALLOW BUILD
```

APPENDIX C – FINAL VERIFICATION CHECKLIST

· Shift-Left Secure SDLC concept implemented
· Vulnerability detection performed
· Vulnerabilities classified by severity
· CVSS analysis included
· SAST simulation included
· Dependency/secret scanning concept included · Policy-as-Code rules defined
· Security gate implemented
· Insecure build blocked
· Remediation performed
· Re-testing performed
· Secure build allowed
· Before/after comparison performed · Performance evaluation included · Results and observations documented · DevSecOps/CI-CD integration explained · Future enhancements included
· References included